LAB / INTERRUPTS

1. OVERVIEW

1.1 Goal

The purpose of this laboratory is to put your theoretical knowledge of interrupts into practice.

1.2 The STM32 external interrupts (EXTI)

In this laboratory, we only want to practice the EXTI interrupts. There are 16 EXTI interrupts available depending on the pin number we choose.

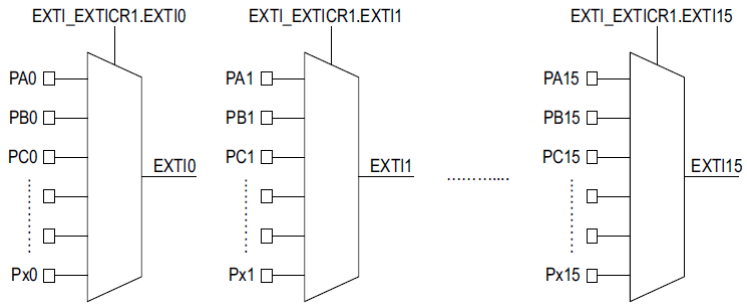


Figure 1: The multiplexer interrupt logic

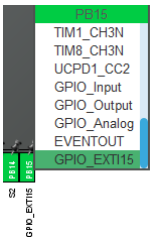
1. TASK ONE

1.1 Goal

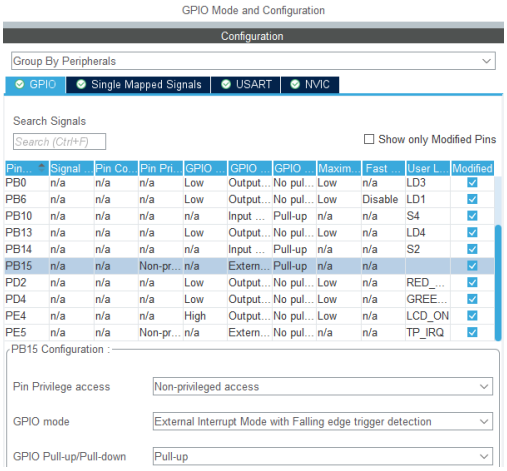
The button S1 will be used now in interruption. The three other buttons are not used now.

The interruption routine will simply invert the state of the led LD1.

To change the input pin as interrupt, you need to select the corresponding input and choose the GPIO_EXTI mode.



You must change now the GPIO configuration to have a pull-up and you can choose the interrupt mode (rising, falling or both).



In the NVIC, disable the Line1, 4 and 5 interrupt and enable the EXTI line 15 interrupt.



Once the code is generated, have a look at the file `stm32u5xxit.c` to look at what has been written. You must now write the code for the interrupt callback function: `HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin)`. Do you see a problem on the button S1 led ?

2. TASK TWO

2.1 Goal

Interrupts are not working as expected. In fact, the buttons bounce. We will solve this problem with software. The idea is to disable this interruption when the interruption arrives. We will get the time when this interruption arrives and we can re-enable it in the main loop after, for example, 100 milliseconds. You must use the HAL functions below:

- `HAL_NVIC_DisableIRQ(EXTI15_IRQn);`
- `HAL_NVIC_EnableIRQ(EXTI15_IRQn);`
- `HAL_GetTick();`

Test your code. Is it working now? Have a look at the interrupt routine `HAL_GPIO_EXTI_IRQHandler()` and try to understand and to solve your problem. Implement the same code for the second button. The leds LD3 and LD4 will reflect the state of the buttons S3 and S4 (with interrupts too). Finally, in the main loop, write the code to have the green led called GREEN_LED blinking at 1 Hz.

3. TASK THREE

3.1 Goal

To reduce power consumption, we can put the microcontroller in sleep mode. On the main loop call the function below to enter sleep mode.

```
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON,  PWR_SLEEPENTRY_WFI)
```

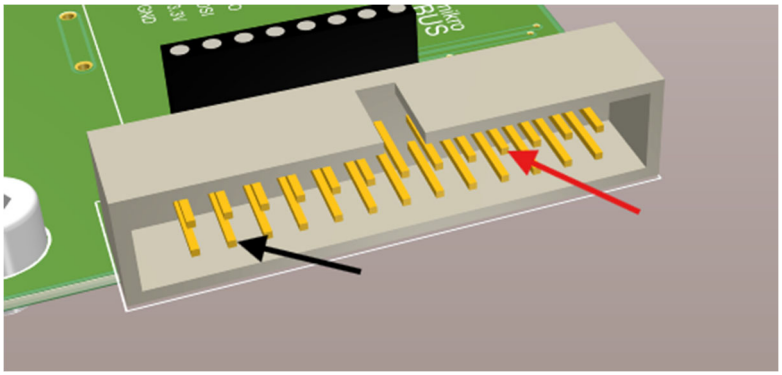
What appends now ? Are the buttons working as expected ? Is the green led blinking as expected and why?

Finally, we want to forget the SysTick interrupt to reduce the consumption at the maximum. We must stop the SysTick with the function HAL_SuspendTick(). We can call HAL_ResumeTick() when the system is waked again. Try it with this modification and see how the system works. What is the problem and how to solve it ?

1. TASK FOUR

1.1 Goal

To stress the microcontroller, we want now to have interruptions from an external input. For that, we will use pin 7 of the extension connector J1 (red arrow). The ground is on the pin 24 (black arrow). The pin 7 is the button S1.



In the interrupt routine, a counter will be incremented.

On this interrupt entry, we will inject signal from the AnalogDiscovery. Try it with 10Hz, 100Hz, 1kHz, 10kHz, 100kHz, 1MHz and 2MHz.

To see the frequency, in the main loop, set a breakpoint when one second elapsed and see the counter variable.

What values of frequency do you find for these frequencies?

Your error comes from the precision of the SysTick. To increase this precision, in the RCC module (graphical view), activate the external crystal @ 16MHz. Select it in the Clock Configuration view. Resolve the CPU frequency for 160MHz.

Test your system again and observe what has changed.